How Paging Works?

Paging is a method used by operating systems to manage memory efficiently. It breaks physical memory into fixed-size blocks called “frames” and logical memory into blocks of the same size called “pages.” When a program runs, its pages are loaded into any available frames in the physical memory.

This approach prevents fragmentation issues by keeping memory allocation uniform. Each program has a page table, which the operating system uses to keep track of where each page is stored in physical memory. When a program accesses data, the system uses this table to convert the program’s address into a physical memory address.

Paging allows for better memory use and makes it easier to manage. It also supports virtual memory, letting parts of programs be stored on disk and loaded into memory only when needed. This way, even large programs can run without fitting entirely into main memory.

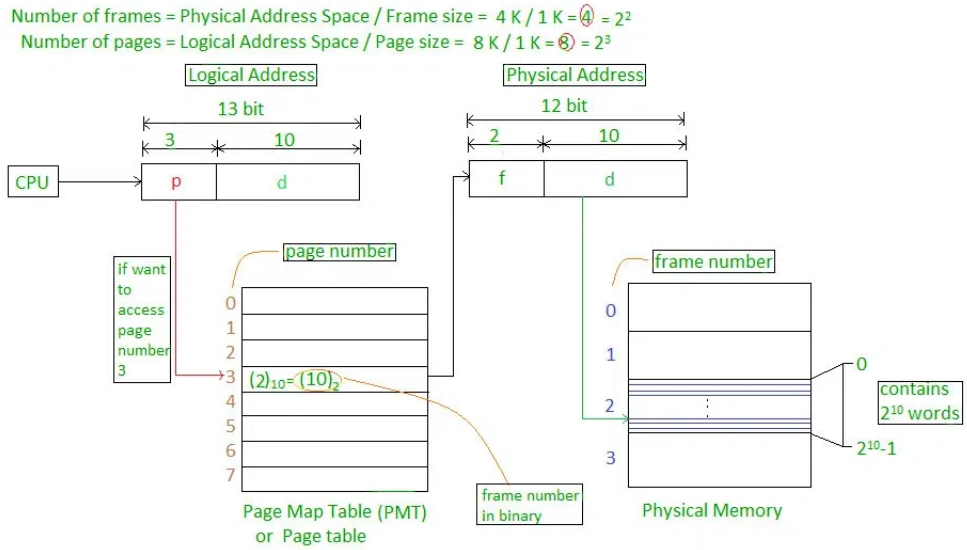
- If Logical Address = 31 bit, then Logical Address Space = 231 words = 2 G words (1 G = 230)
- If Logical Address Space = 128 M words = 27 * 220 words, then Logical Address = log2 227 = 27 bits
- If Physical Address = 22 bit, then Physical Address Space = 222 words = 4 M words (1 M = 220)
- If Physical Address Space = 16 M words = 24 * 220 words, then Physical Address = log2 224 = 24 bits

The mapping from virtual to physical address is done by the Memory Management Unit (MMU) which is a hardware device and this mapping is known as the paging technique.

- The Physical Address Space is conceptually divided into a number of fixed-size blocks, called **frames**.
- The Logical Address Space is also split into fixed-size blocks, called **pages**.
- Page Size = Frame Size

Example

- Physical Address = 12 bits, then Physical Address Space = 4 K words
- Logical Address = 13 bits, then Logical Address Space = 8 K words
- Page size = frame size = 1 K words (assumption)



The address generated by the CPU is divided into

- **Page number(p):** Number of bits required to represent the pages in Logical Address Space or Page number
- **Page offset(d):** Number of bits required to represent a particular word in a page or page size of Logical Address Space or word number of a page or page offset.